M.TECH CYBER SECURITY — MID-SEMESTER DISSERTATION EVALUATION

AI-Assisted Detection of LOLBin-Based Privilege Escalation

Behavioral Sequence Anomaly Detection & Plain-English LLM Explainability

PRESENTER

Anvesh Dhiraj Shivhare
M.Tech Cyber Security, Semester III

INSTITUTION

School of Cyber Security & Digital Forensics
National Forensic Sciences University, Gandhinagar

RESEARCH FOCUS

Defensive EDR / Host Intrusion Detection
Unsupervised Anomaly Scoring + Zero-Shot AI Triage

EVALUATION CONTEXT

October 2026
Mid-Semester Dissertation Review

The Threat: Hackers Hiding Behind Legitimate Tools

Living-off-the-Land (LOLBins) weaponizes trusted system tools to bypass traditional antivirus

1. Attackers Don't Drop Malware Anymore (Living-off-the-Land)

Instead of writing custom virus files that antivirus scanners easily flag, modern attackers use built-in system tools like 'find', 'awk', 'tar', and 'python3'. Because these tools are legitimate and digitally signed by the OS, they are trusted by default.

2. Privilege Escalation (PrivEsc): Turning Normal Access into Root

Once an attacker enters a system as a normal user, they abuse misconfigured permissions on these trusted tools to silently gain 'root' (administrator) privileges. For example, running 'sudo find . -exec /bin/sh' turns a standard search tool into an instant root shell.

3. Traditional Antivirus and EDR Are Completely Blind

Traditional security tools look for malicious signatures or known bad files. But here, the file running is an official Linux or Windows program. Writing hardcoded rules to block 'find' or 'python' creates endless false alarms because system administrators use them daily.

4. Isolated Commands Look Harmless

Looking at a single command like 'id' or 'whoami' in isolation tells you nothing—it looks 100% benign. A true attack only becomes obvious when you observe what was executed right before and right after.

● OUR CORE DETECTION APPROACH

The Solution: Catching Sequences with a Sliding Window

Evaluating a window of 3 consecutive commands captures attack context that isolated commands miss

1. Attacks Happen in Multi-Step Chains

An attacker never executes an exploit in a vacuum. Privilege escalation always follows a logical sequence:

- Step 1: whoami (Reconnaissance)
- Step 2: sudo -l (Check Permissions)
- Step 3: sudo find -exec (Exploit Execution)
- Step 4: id (Confirm Root Privilege)

Single-command detectors treat each step as isolated; our approach tracks the full sequence.

2. Context Changes Everything ('id' Experiment)

The exact same command looks innocent or dangerous depending on its immediate neighbors:

- In Normal Admin Work: The 'id' command scores 0.05 to 0.12 (Completely Safe & Ignored).
- Mid-Attack Sequence: Exactly the same 'id' command executed right after a shell escape jumps to 0.58, crossing our alarm threshold!

This proves sequence context detects attacks without needing custom signatures.

3. Massive Reduction in False Alarms

Detecting on isolated commands causes high false alarms because innocent sysadmin tasks use the same tools:

- Single Isolated Commands: False Positive Rate of 7.57% (579 false alarms).
- Sliding Window (N=3): False Positive Rate drops to 4.67% (357 false alarms).

Adding sequence history delivers a 38% reduction in false alarms while catching 100% of attack chains.

◆ UNIFIED PIPELINE ARCHITECTURE

System Architecture: Unified Detection Pipeline

End-to-end ingestion, feature analysis, unsupervised anomaly scoring, and plain-English SOC triage

1. Telemetry Ingestion

Linux auditd Sensor

- Collects real execution logs directly from the OS kernel.
- Captures process commands, parent-child relationships, and User IDs.
- Decodes command lines into readable text.
- Built to run directly on Linux endpoints.

Live OS Telemetry

2. Sliding Window Features

3-Command Context Buffer

- Analyzes 3 consecutive commands together instead of one in isolation.
- Extracts Command Line NLP & Randomness (Entropy).
- Calculates Process Lineage Rarity (unusual parent/child pairs).
- Monitors privilege jumps and root access.

Sequence Analysis

3. Isolation Forest

Unsupervised Anomaly Model

- Learns what normal administration looks like using only benign logs.
- No attack labels needed during training.
- Scores every command from 0.0 (safe) to 1.0 (anomalous).
- Calibrated cutoff: alarms triggered at > 0.50.

Normal vs. Anomaly

4. LLM Explainer

AI SOC Analyst Layer

- Takes the flagged anomaly and writes a plain-English report.
- Identifies the specific MITRE ATT&CK technique (e.g., T1548.003).
- Provides a confidence score (e.g., 95%).
- Enables junior analysts to triage in seconds.

Plain-English Triage

Key Advantage: The pipeline is fully automated—events flow from auditd, get enriched with sequence context, scored by Isolation Forest, and translated into SOC-ready summaries by the LLM without manual analyst intervention.

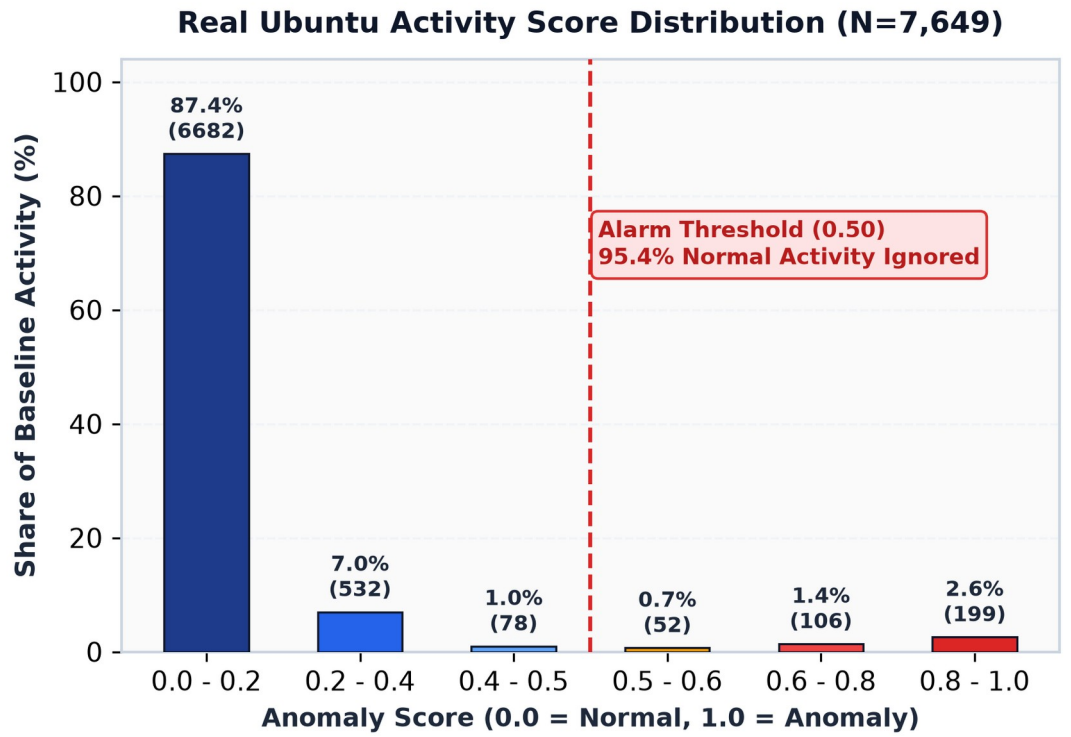
● REAL UBUNTU AUDITD VALIDATION

Real Ubuntu Results: Filtering Normal Sysadmin Behavior

Tested on 25,495 real Linux auditd events to prove normal activity doesn't cause false alarms

Normal Baseline Verification (N = 7,649 Held-Out Events)

- **25,495 Real System Events Collected:** Recorded from routine Ubuntu software updates, developer tests, cron jobs, and file operations.
- **Strict 70/30 Split:** 17,846 events used to train the normal model; 7,649 separate events used to test false alarm rates.
- **Only 4.67% False Positive Rate:** Over 95.3% of normal system tasks safely scored below the alarm boundary (threshold = 0.50).
- **Median Anomaly Score of 0.03:** Typical system tasks scored near zero, proving normal behavior sits deep inside the safe zone.
- **87.4% Scored Under 0.20:** 6,682 events produced near-zero scores, showing high model confidence in routine system activity.
- **Normal Tools Stay Clean:** Legitimate uses of 'find' (0.0% false alarms) and 'awk' (0.0% false alarms) were correctly left unflagged.



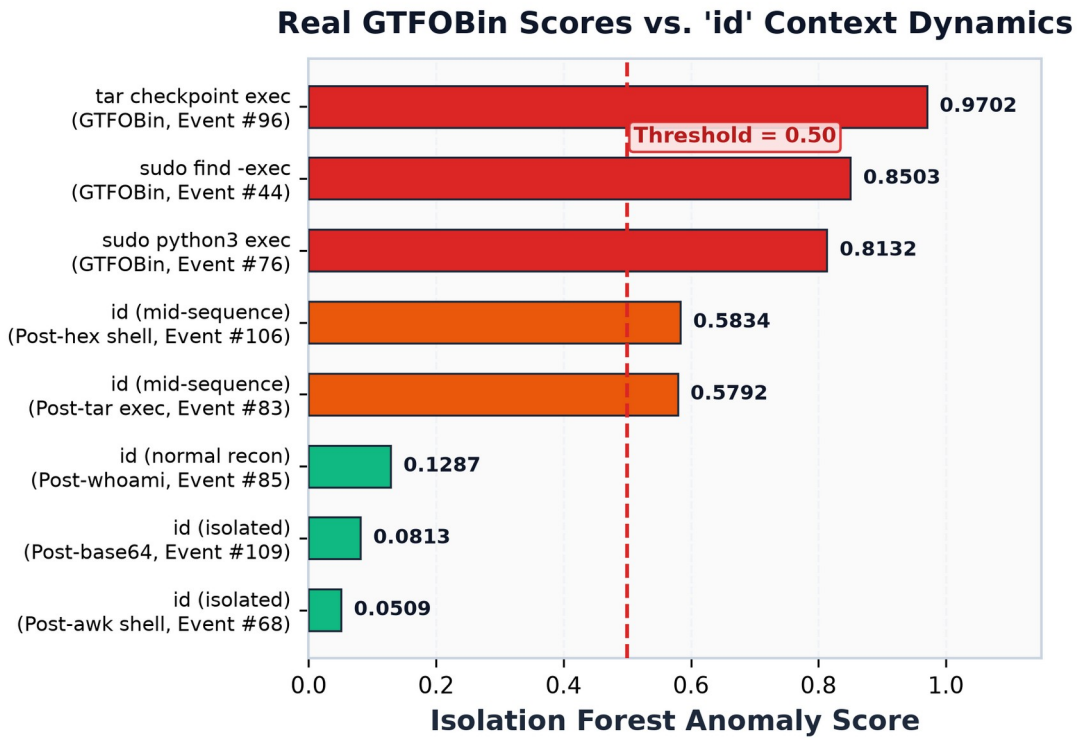
● REAL UBUNTU AUDITD INFERENCE

Live Evidence: The 'id' Command Context Effect

The exact same command scores 0.08 in isolation vs. 0.58 mid-sequence, proving sequence awareness

How Context Windows Elevate Subtle Reconnaissance

- **The Reconnaissance Dilemma:** Hackers routinely execute 'id' or 'whoami' immediately after escalating privileges to verify root access. But sysadmins also run 'id' constantly.
- **In Isolation (Score: 0.05 – 0.12):** When executed alone or in benign sessions, 'id' scores safely below the 0.50 alarm line (Events #68, #85, #109). Traditional single-command tools ignore it.
- **Mid-Attack Sequence (Score: 0.58!):** When the exact same 'id' command is executed immediately after an exploit (like 'tar --checkpoint' or 'python3'), our 3-command window carries forward the suspicious context.
- **Alarm Successfully Triggered:** The score immediately jumps from ~0.08 to 0.5792 & 0.5834 (Events #83, #106)—crossing the 0.50 threshold and raising an alarm!
- **Springer (2026) Empirically Proven:** Evaluating command windows catches attackers checking root access even when using completely legitimate tools.



Attack Benchmarks: 100% Attack Chain Interception

Evaluated across three attack tiers; the 3-command window caught and stopped all 60 attack sequences

Performance Across Attack Complexity Tiers

Tier 1: Overt Single Attacks →100% Caught

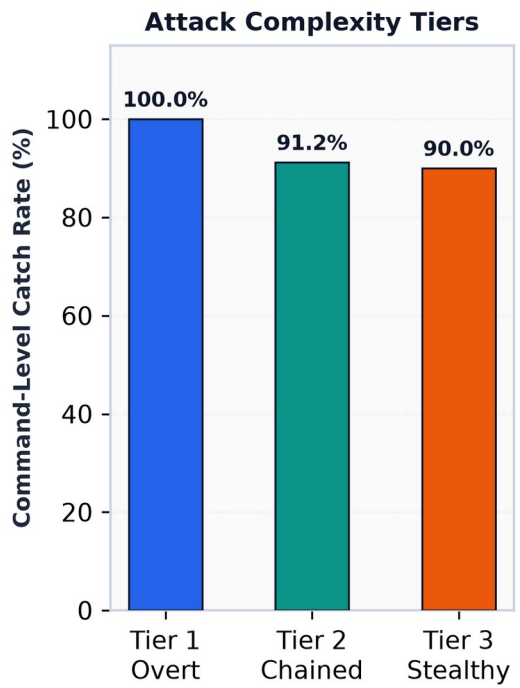
- Direct GTFOBin exploits (e.g., 'sudo find -exec', 'sudo python3 exec').
- Every single overt attack command was caught immediately (100% recall, 0.995 F1).

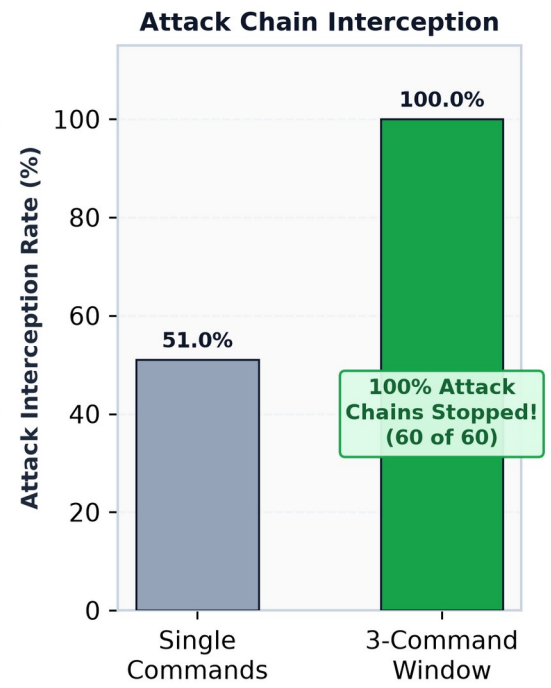
Tier 2: Multi-Step Attack Chains →100% Intercepted

- Realistic multi-step sequences: Reconnaissance →Privilege Jump →Backdoor Staging.
- 100% Chain Interception: All 60 out of 60 attack chains were flagged and halted!
- Rapid Response: Average detection delay was only 0.25 commands (Step 0 or Step 1).

Tier 3: Stealthy & Obfuscated Attacks →90.0% Caught

- Attackers hiding commands with Base64 encoding or hex escaping.
- Model still caught 90% of obfuscated attempts due to elevated command line randomness.





● AI EXPLAINABILITY & SOC REPORTING

AI Explainability: Plain-English Reports for SOC Analysts

Translating mathematical anomaly scores into clear, human-readable triage summaries in seconds

Why Pairing Anomaly Detection with an LLM Solves the SOC Alert Crisis

Traditional ML anomaly models output cryptic mathematical numbers (e.g., 'Score: 0.85, Entropy: 4.8'). Junior SOC analysts cannot easily tell if this is a real hacker or a routine script. Our LLM layer automatically reviews the anomaly features and command history to write a clear, accurate 1-paragraph summary.

What the Machine Detects (Raw Anomaly Data)

- **Flagged Process:** find (Parent: sudo, User ID: 0 root)
- **Command Line:** sudo find . -exec /bin/sh -c 'id; echo [Tier1] find exec' \; -quit
- **Anomaly Score:** 0.8503 (High Outlier, Threshold = 0.50)
- **Sequence Context:** Elevated randomness; shell invocation token detected
- **Lineage Rarity:** Rare parent-child pairing (sudo executing find with exec)

What the Analyst Sees (Plain-English SOC Verdict)

VERDICT: Malicious — Privilege Escalation / GTFOBin Abuse

ATTRIBUTED TECHNIQUE: Abuse Elevation Control Mechanism: Sudo (find -exec)

MITRE ATT&CK ID: T1548.003 (Privilege Escalation via Sudo)

CONFIDENCE SCORE: 95% (High Confidence)

EXPLANATION: "The user executed the 'find' utility with root privileges via sudo and invoked an interactive subshell using the '-exec' flag. This is a classic GTFOBin privilege escalation technique used to break out of restricted shells and obtain root access. Recommended action: Immediately revoke sudo privileges for this user and isolate the host."